

DSS Evakuasi Bencana Berbasis Graph dan Dijkstra

Anggota:

- I Gede Ega Sanjaya Putra (2501010099)
- Dewa Gede Reihan Waiswana (2501010122)
- I Ketut Kayana Pradithya (2501010112)

Teknologi utama:

- **Python**
- **Streamlit**
- **Folium**
- **OpenRouteService API**
- **Graph Theory**



Studi Kasus — Tantangan & Solusi



Permasalahan

- Masyarakat sering kesulitan menentukan jalur evakuasi terbaik
- Beberapa jalan rusak atau berbahaya pascabencana
- Jalur terdekat belum tentu yang paling aman
- Pengambilan keputusan harus cepat dan berbasis data

Solusi

Membangun DSS yang memodelkan lokasi sebagai graph, menghitung rute optimal dengan mempertimbangkan jarak dan risiko, serta menampilkan hasilnya pada peta interaktif agar rekomendasi lebih cepat dan dapat diandalkan.

Model Graph — Representasi dan Bobot

Adjacency List

Graph direpresentasikan dengan **adjacency list**, yaitu setiap node menyimpan daftar node yang terhubung beserta bobotnya.

Weighted Graph

Setiap jalur memiliki **bobot** yang merepresentasikan jarak atau tingkat risiko pada jalur evakuasi.

Directed Graph

Graph bersifat **directed**, sehingga arah setiap jalur harus diperhatikan dalam perhitungan rute.

Algoritma: Dijkstra

Tujuan: menemukan jalur dengan total bobot terkecil dari sumber ke tujuan pada weighted graph dengan bobot non-negatif.

01

1. Inisialisasi

Tetapkan jarak semua node = ∞ , kecuali node awal = 0, lalu tandai semua node belum dikunjungi.

02

2. Pilih node aktif

Pilih node belum dikunjungi dengan jarak terkecil sebagai node aktif.

03

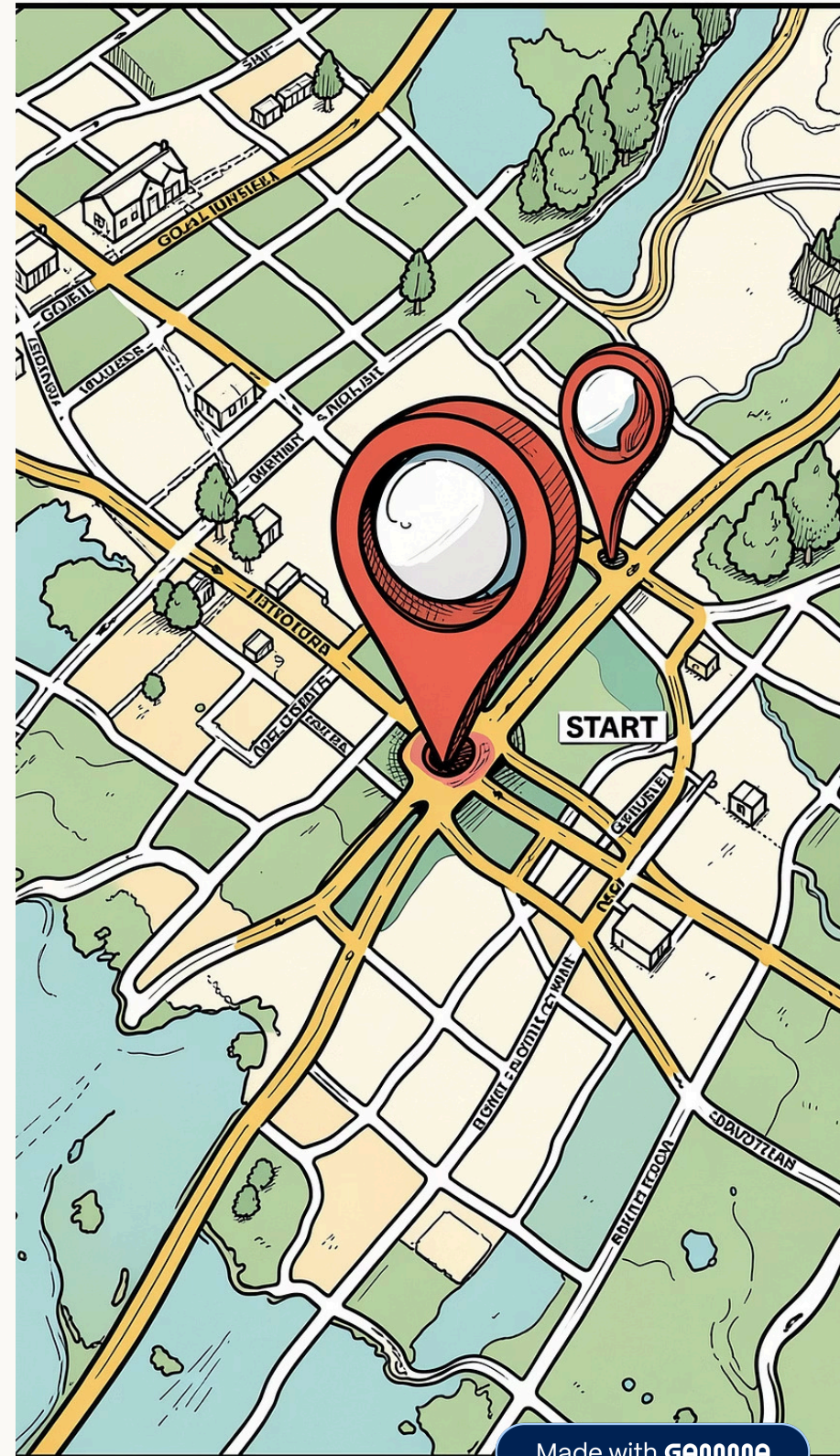
3. Relaksi tetangga

Perbarui jarak tetangga melalui node aktif jika ditemukan jalur yang lebih pendek.

04

4. Ulangi hingga selesai

Teruskan hingga node tujuan diproses atau semua node telah dikunjungi.



Mengapa memilih Dijkstra?



Mudah Diimplementasikan

Kode ringkas dan didukung banyak pustaka seperti `networkx` dan `heapq`.



Efisien

Semakin banyak lokasi dan jalur, waktu proses akan bertambah, tetapi Dijkstra tetap cukup cepat karena menggunakan struktur data yang efisien.



Cocok untuk Weighted Graph

Menangani bobot jalan, seperti jarak dan risiko, secara langsung.

Alternatif Singkat

- **BFS** — untuk graf tak berbobot, sehingga tidak cocok di sini
- **DFS** — berguna untuk eksplorasi, bukan pencarian jalur optimal
- **A*** — A* tidak dipilih karena membutuhkan heuristik atau estimasi jarak menuju tujuan, sedangkan sistem ini menggunakan data bobot aktual sehingga Dijkstra sudah cukup efektif.



Untuk rute evakuasi komposit yang mempertimbangkan jarak dan risiko, Dijkstra memberikan solusi deterministik tanpa memerlukan heuristik yang kompleks.

Arsitektur Sistem

Alur utama: Dataset → Graph Builder → Dijkstra → Recommendation Engine → Map Visualization → User Interface

Dataset

Berisi node, edge, atribut risiko, kondisi jalan, dan koordinat.

Graph Builder

Membangun adjacency list atau matrix, lalu menghitung bobot komposit.

Dijkstra

Menentukan jalur terpendek berdasarkan biaya minimum antara sumber dan target.

OpenRouteService

Mensinkronkan rute dengan jaringan jalan nyata untuk meningkatkan akurasi visual.

Folium / Streamlit

Menampilkan peta interaktif dan antarmuka pengguna.

Fitur Sistem



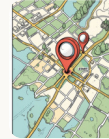
Tambah Lokasi

Form untuk menambahkan node baru, termasuk tipe, koordinat, dan kapasitas.



Tambah Jalur

Menambahkan edge dengan atribut jarak, kondisi jalan, dan skor risiko.



Titik Awal & Tujuan

Pengguna memilih titik sumber dan tujuan langsung pada peta.



Heatmap Area Bahaya

Menampilkan intensitas risiko untuk membantu pengambilan keputusan cepat.



API Routing & Skoring Risiko Adaptif

Integrasi OpenRouteService dengan model penilaian risiko adaptif berbasis data lapangan.

Hasil Analisis — Dampak Sistem

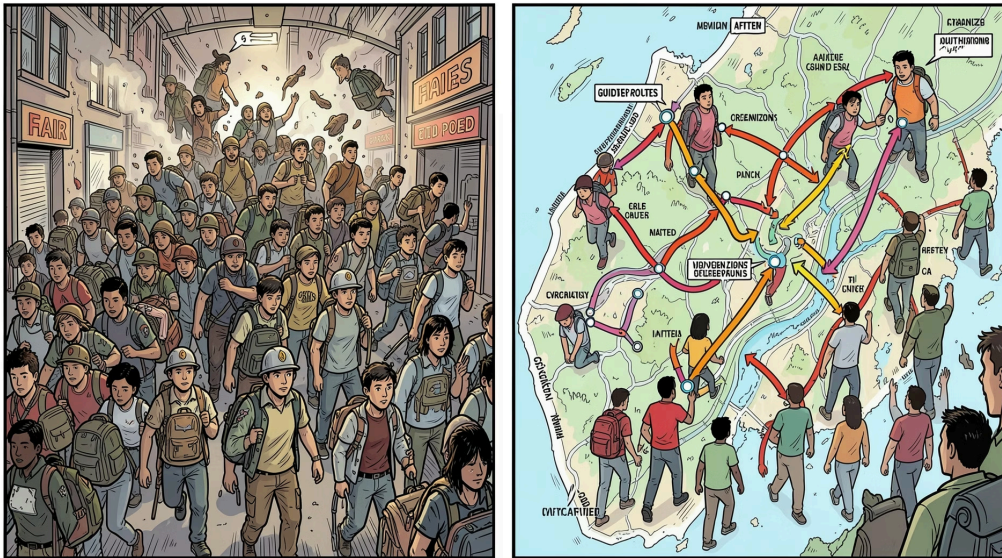
Sebelum DSS

- Rute dipilih secara manual dan subjektif
- Risiko melintasi area berbahaya lebih tinggi
- Waktu evakuasi belum optimal

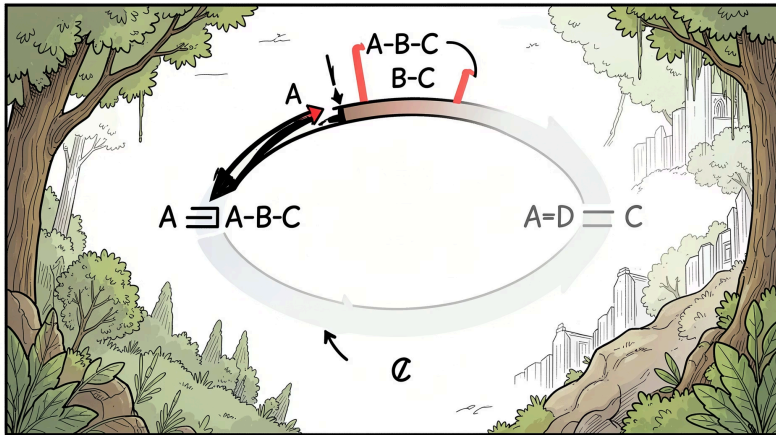
Setelah DSS

- Rute dihitung otomatis berdasarkan biaya, yaitu jarak dan risiko
- Sistem memberikan rekomendasi interaktif beserta alternatif rute
- Keselamatan meningkat dan waktu evakuasi lebih efisien

- ✔ DSS mengurangi ketergantungan pada keputusan ad hoc dan menyediakan bukti visual untuk mendukung rekomendasi rute.



Contoh Hasil Perhitungan



Contoh jalur:

- $A \rightarrow B \rightarrow C$ – total bobot 6
- $A \rightarrow D \rightarrow C$ – total bobot 11

Kesimpulan: Dijkstra memilih $A \rightarrow B \rightarrow C$ karena biaya totalnya lebih kecil (6 vs 11). Sistem juga menyimpan jalur alternatif dan metadata seperti jarak serta risiko pada setiap edge untuk kebutuhan audit dan fallback.

- ❏ Sistem menampilkan jalur utama dan alternatif, beserta alasannya, misalnya untuk menghindari zona berisiko.

Kelebihan Sistem

- Menentukan Jalur Evakuasi Terbaik
- Visualisasi Interaktif
- Mempertimbangkan Risiko
- Fleksibel dan Mudah Dikembangkan
- Integrasi API Routing

Kekurangan Sistem

- Bergantung pada Data yang Dimasukkan
- Belum Menggunakan Data Real-Time
- Belum Mendukung Multi-User
- Belum Menggunakan Machine Learning
- Ketergantungan pada Internet untuk API Routing

Kendala, Solusi, dan Kesimpulan

Route hilang setelah klik

Solusi: gunakan Session State pada Streamlit untuk mempertahankan tampilan.

Jalur tidak mengikuti jalan nyata

Solusi: sinkronkan hasil Dijkstra dengan OpenRouteService agar rute menyesuaikan jaringan jalan.

Node baru tidak terhubung

Solusi: validasi topologi, yaitu wajib menambahkan minimal 1 edge saat membuat node baru.

Error data graph

Solusi: validasi input dan sanitasi atribut, seperti jarak ≥ 0 dan risk score dalam skala yang terdefinisi.

Visualisasi kurang menarik

Solusi: kombinasikan Folium, heatmap, dan custom markers untuk meningkatkan kontras dan keterbacaan.

Kesimpulan

1. DSS berhasil membantu menentukan jalur evakuasi optimal berbasis graph.
2. Graph, terdiri dari node dan edge, efektif memodelkan lokasi serta kondisi jalan.
3. Dijkstra cocok untuk mencari rute dengan biaya minimum pada graf berbobot non-negatif.
4. OpenRouteService meningkatkan akurasi rute agar sesuai dengan jaringan jalan nyata.
5. Sistem menyediakan rekomendasi interaktif yang dapat diandalkan untuk evakuasi.

i Langkah selanjutnya: integrasi data real-time dari sensor dan laporan warga, evaluasi A* dengan heuristik geografis, serta pengujian lapangan skala kecil.